Kubernetes Controllers - Student Tasks

Task 1: Create and Delete Standalone Pod

Objective: Understand that standalone pods are unmanaged

```bash
# TODO: Create a standalone pod with Apache (httpd image)
kubectl run apache-pod --image=httpd

# TODO: List all pods
kubectl get pods

# TODO: Delete the pod
kubectl delete pod apache-pod

# TODO: List pods again - verify it's gone
kubectl get pods

# QUESTION: Where did the pod go? Can you get it back?
```

Expected Answer: Pod is completely deleted. No controller to recreate it.

Task 2: Create Deployment and Scale It

Objective: Learn how controllers maintain desired state

```bash
```

```bash
# TODO: Create a deployment with 2 replicas
kubectl create deployment web --image=nginx --replicas=2

# TODO: Check how many pods exist
kubectl get pods

# TODO: Scale the deployment to 5 pods
kubectl scale deployment web --replicas=5

# TODO: Check pods again
kubectl get pods

# TODO: Scale back to 2 pods
kubectl scale deployment web --replicas=2

# TODO: Check final pod count
kubectl get pods

# QUESTION: How many pods do you see now?
# QUESTION: What happens to the pods when you scale down?
```

Expected Answer: 2 pods. Extra pods are terminated.

Task 3: Pod Self-Healing

Objective: Prove that Deployment recreates failed pods

```bash
```

```bash
# TODO: Create a deployment with 3 replicas
kubectl create deployment app --image=nginx --replicas=3

# TODO: List all pods and copy one pod name
kubectl get pods
# Copy the first pod name from the output

# TODO: Delete that pod
kubectl delete pod <PASTE-POD-NAME-HERE>

# TODO: Immediately list pods again (within 5 seconds)
kubectl get pods

# TODO: Wait 10 seconds and list again
sleep 10
kubectl get pods

# QUESTION: What happened to the deleted pod?
# QUESTION: Is the pod count still 3?
```

Expected Answer: Deployment automatically created a new pod. Pod count stayed at 3.

Task 4: Compare Controllers

Objective: Understand different controller types

```bash
```

```
# Part A: Deployment (you've used this)
kubectl create deployment myapp --image=nginx --replicas=2
kubectl get deployment myapp
kubectl get pods -l app=myapp

# Part B: Create a ReplicaSet
cat > rs.yaml << 'EOF'
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myrs
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myrs
  template:
    metadata:
      labels:
        app: myrs
    spec:
      containers:
      - name: nginx
        image: nginx
EOF

# TODO: Apply the ReplicaSet
kubectl apply -f rs.yaml

# TODO: List ReplicaSets
kubectl get replicaset

# TODO: List pods created by ReplicaSet
kubectl get pods -l app=myrs

# TODO: Delete one pod from ReplicaSet
kubectl delete pod $(kubectl get pods -l app=myrs -o jsonpath='{.items[0].metadata.n

# TODO: Check pods again
kubectl get pods -l app=myrs

# QUESTION: What's the difference between Deployment and ReplicaSet?
```

Expected Answer: Both maintain pod count. Deployment is higher-level and better for updates.

Task 5: Update Deployment Image

Objective: Learn how to update apps without downtime

```bash
# TODO: Create deployment with nginx:1.19
kubectl create deployment website --image=nginx:1.19

# TODO: Check the image version
kubectl describe deployment website | grep Image

# TODO: Update to nginx:alpine
kubectl set image deployment/website website=nginx:alpine

# TODO: Check image again
kubectl describe deployment website | grep Image

# TODO: List pods - notice new ones are being created
kubectl get pods

# QUESTION: Did you lose access to your app during the update?
```

Expected Answer: No downtime. Pods were updated one by one.

Task 6: Debug and View Pod Details

Objective: Learn how to inspect and troubleshoot

```bash
```

```bash
# TODO: Create a deployment
kubectl create deployment test --image=nginx --replicas=2

# TODO: Get detailed info about deployment
kubectl describe deployment test

# TODO: Get detailed info about a specific pod
kubectl get pods
# Copy a pod name
kubectl describe pod <POD-NAME>

# TODO: View pod logs
kubectl logs <POD-NAME>

# TODO: Get more details (CPU, memory, node)
kubectl get pods -o wide

# QUESTION: Which node is each pod running on?
# QUESTION: What's the pod's IP address?
```

Expected Answer: Pods have their own IP, running on nodes, logs show container output.

Task 7: Create Your Own Deployment

Objective: Apply everything learned

```bash
```

```
# TODO: Create a deployment called "myservice"
# Requirements:
#   - Image: httpd (Apache)
#   - 3 replicas
#   - Label it with app=myservice

# Hint: Use kubectl create deployment command

# TODO: Verify 3 pods are running
kubectl get pods

# TODO: Scale it to 5 pods
# (use kubectl scale command)

# TODO: Delete 1 pod
# (use kubectl delete pod command)

# TODO: Verify it auto-recreates (5 pods still exist)

# TODO: Scale back to 1 pod

# TODO: Describe the deployment to see details
kubectl describe deployment myservice

# TODO: Delete the deployment
kubectl delete deployment myservice

# QUESTION: When you delete the deployment, what happens to the pods?
```

Expected Answer: All pods are deleted when deployment is deleted.

Task 8: Namespace Management

Objective: Organize resources with namespaces

```bash
```

```
# TODO: Create 2 new namespaces
kubectl create ns dev
kubectl create ns prod

# TODO: Create deployments in each namespace
kubectl -n dev create deployment app --image=nginx --replicas=2
kubectl -n prod create deployment app --image=nginx --replicas=3

# TODO: List pods in dev namespace only
kubectl get pods -n dev

# TODO: List pods in prod namespace only
kubectl get pods -n prod

# TODO: List all pods across all namespaces
kubectl get pods -A

# TODO: Switch context to dev namespace
kubectl config set-context --current --namespace=dev

# TODO: List pods (should show dev pods by default)
kubectl get pods

# TODO: Delete dev namespace (deletes all its resources)
kubectl delete ns dev

# TODO: Verify dev is gone but prod remains
kubectl get ns
kubectl get pods -n prod

# QUESTION: Why are namespaces useful?
```

Expected Answer: Organize resources, isolate environments (dev/prod).

Summary Questions (Answer these)

1. **What's the main difference between standalone Pod and Deployment?**
   - Standalone pods don't auto-recreate when deleted
   - Deployments auto-recreate pods to maintain desired state
2. **How do you scale a Deployment?**
   - `kubectl scale deployment <name> --replicas=<number>`
3. **If you delete a pod in a Deployment, what happens?**

- Deployment controller sees desired state doesn't match current state
- It automatically creates a new pod

4. **Which is better for production: Standalone Pod or Deployment?**
   - Deployment (or other controllers)
5. **What's a ReplicaSet?**
   - Lower-level controller that maintains pod replicas
   - Deployments use ReplicaSets internally
6. **How do you update the image in a Deployment?**
   - `kubectl set image deployment/<name> <container>=<image>`
7. **What happens to pods when you delete a Deployment?**
   - All pods managed by that Deployment are deleted
8. **Why use namespaces?**
   - Organize resources
   - Separate dev/staging/prod environments
   - Isolate projects